OpenSeesPy CFRP ply property generator: chat transcript

Claude Code session (Claude desktop app), Fri 23 Oct 2026, 11:37 to Mon 26 Oct 2026, 09:37 (Pacific time).
Working folder: ME295_project/Code for Project/10-23-26 (Tara).
Shown: your messages, Claude's replies, questions and recaps. Tool calls appear as one-line summaries (▸); their outputs, Claude's internal reasoning and system notices are omitted.

You Fri 23 Oct 2026, 11:37

I want to use openseespy to model the large (geometrically nonlinear) deformation of a thin-shell beam made up of a CFRP laminate with multiple layers in different directions. I want to specify the material properties of the laminate to do so. Generate an agent that:

1. Takes user input of the material properties of a transversely isotropic CFRP ply (E1, E2, nu12, G12,nu23). For a transversely isotropic material, fill in the rest of the material parameters by assuming E2 = E3, G12 = G13, nu12 = nu13, and G23 = E2/(2*(1+nu23)). If no nu23 is given, assume it is equal to nu12 but output a note in the chat saying that this assumption was made.
2. Uses the material properties to define the material properties in OpenSeesPy in the material principal axes with the ElasticOrthotropic function.

Call this agent PlyPropertyGenerator

▸ Bash: List repo files and check openseespy install
▸ Bash: Read sibling env and model code
▸ Bash: Find shells env python and plate material setup
▸ PowerShell: Locate conda envs with Python
▸ PowerShell: Search for a python executable

Claude

Found the project's `shells` Python env. Next I'll check numerically which Poisson's ratio OpenSees expects in the `vzx` slot of `ElasticOrthotropic`.

▸ Bash: Test ElasticOrthotropic Poisson convention on one brick

Claude

Confirmed: OpenSees's `vzx` slot is ν31 (x-strain from z-stress), not ν13. That means the ply's third Poisson's ratio has to be passed as ν31 = ν13·E3/E1 = ν12·E2/E1. Now writing the module.

▸ Write: Code for Project\10-23-26 (Tara)\ply_properties.py

Claude

Now the agent definition, at the repo root so it's available across the project.

▸ Write: .claude\agents\PlyPropertyGenerator.md
▸ Bash: Run module on sample ply and verify with brick tests

Claude

Script typo (OpenSees DOFs are 1-based); fixing and rerunning.

▸ Bash: Rerun brick verification

Claude

Material verified in all three directions; the shell-wrapper check just needs ≥3 layers.

▸ Bash: Rerun shell wrapper sanity check

Recap: PlyPropertyGenerator agent and ply_properties.py are written. I checked them on a sample CFRP ply and all nine elastic constants came back correctly from OpenSees. Nothing is committed yet.

Claude

I created the **PlyPropertyGenerator** agent. On a sample CFRP ply, the material it defines gives back all the moduli and Poisson's ratios you'd expect in OpenSeesPy.

What was created

- `.claude/agents/PlyPropertyGenerator.md`: the agent itself. It reads E1, E2, nu12 and G12 from your request (nu23, density and material tag are optional) and runs the script below. It reports every constant and whether you supplied it or it was derived. If nu23 is missing, it adds a visible note that nu23 = nu12 was assumed. It also gives you the `ops.nDMaterial(...)` line. If a required input is missing it stops rather than guessing, and it converts GPa to MPa to match the project's N/mm/MPa units.
- `ply_properties.py`: the calculation, which you can also run on its own:
  - `ply_properties(E1, E2, nu12, G12, nu23=None, rho=0)` fills in E3 = E2, G13 = G12, nu13 = nu12 and G23 = E2/(2(1+nu23)).
  - `define_ply_material(tag, p)` creates the `ElasticOrthotropic` material in the current model.
  - It warns if the constants are physically impossible (the stiffness matrix wouldn't be positive definite).

OpenSees uses ν31, not ν13 The arguments are `Ex, Ey, Ez, vxy, vyz, vzx, Gxy, Gyz, Gzx`. I ran a single-element test, and `vzx` turns out to be the Poisson's ratio for strain in x from a load in z. That means you have to pass **ν31 = ν12·E2/E1** (about 0.022 for CFRP), not ν13 = 0.3. The code does this, and the agent's instructions explain why.

Checks

- With E1 = 135 GPa, E2 = 10 GPa, ν12 = 0.3, G12 = 5 GPa and ν23 = 0.45, a single 3D block element loaded in each of the three directions gave back the right moduli and all six Poisson's ratios.
- Leaving out ν23 printed the note, with ν23 = 0.3 and G23 = 3846 MPa.
- The material also builds without errors as a layered shell element with large-rotation (corotational) formulation (`PlateFiber` → `LayeredShell` → `ASDShellQ4 -corotational`), which is the setup you'll need for the laminate.

Next step for the laminate: the agent only defines the ply in its own fibre axes. Stacking plies at different angles isn't built yet. The standard `LayeredShell` section has no per-layer angle input, so each angled ply needs its own rotated material.

To use it, ask something like "use PlyPropertyGenerator with E1 = 135 GPa, E2 = 10 GPa, nu12 = 0.3, G12 = 5 GPa".

> **You** Fri 23 Oct 2026, 11:41
>
> Don't commit yet.

Claude

Understood, I won't commit anything. The agent definition and `ply_properties.py` are still uncommitted in your working tree, ready for whenever you want to commit them.

> **You** Fri 23 Oct 2026, 11:56
>
> Now add to the capabilities of the PlyPropertyGenerator agent. Ask for user input of the ply angle relative to the global xy coordinate frame, and rotate the matrix of material properties so it is aligned with the global xy coordinate frame. Also add a plane stress and a plate-from-plane-stress wrapper so that the material properties are consistent with a thin plate under plane stress

- ▸ Bash: Probe which nDMaterial types exist

Claude

Native OpenSees has no general anisotropic material, so I'm checking the OpenSeesPy binary for any material or wrapper that can take a ply angle.

- ▸ Bash: Search OpenSees binary for rotation-capable materials
- ▸ Bash: Grep binary strings for orientation-related materials
- ▸ Bash: Grep binary for shell section and wrapper usage strings
- ▸ Bash: Read usage strings for candidate materials
- ▸ Bash: Test ASDShellQ4 -local against rotated Qbar
- ▸ Bash: Rerun without output filter

Claude

Crashed — I was fixing DOFs that I also prescribed with `sp`. Fixing only the non-prescribed DOFs.

- ▸ Bash: Rerun -local membrane test
- ▸ Bash: Capture full output and crash trace
- ▸ Bash: Bisect where OpenSees exits
- ▸ Bash: Bisect analysis steps
- ▸ Bash: Try material chain and solver variants

Claude

`FullGeneral` crashes when every DOF is prescribed; `UmfPack` works, and the 0° result matches Q11·t. Rerunning the angle test with that solver.

- ▸ Bash: Rerun -local membrane test with UmfPack
- ▸ Read: Code for Project\10-23-26 (Tara)\ply_properties.py
- ▸ Write: Code for Project\10-23-26 (Tara)\ply_properties.py
- ▸ Bash: Run updated module and checks at several angles

Claude

The checks pass at every angle tried (0°, ±45°, 90°, 17.3°). Updating the agent instructions.

- ▸ Write: .claude\agents\PlyPropertyGenerator.md
- ▸ Bash: Check missing-angle error and git status

> **Recap:** PlyPropertyGenerator now asks for the ply angle, rotates the stiffness to the global frame, and builds the plane-stress and plate-from-plane-stress wrappers. It is checked against OpenSees at 0°, ±45°, 90°, 30° and 17.3°. Nothing is committed, as you asked.

Claude

PlyPropertyGenerator now asks for the ply angle, rotates the ply stiffness into the global xy frame, and wraps the material for a thin plate. In OpenSees the rotation is applied on the shell element, not in the material, because OpenSees 3.8 has no material that accepts an angle or a full anisotropic stiffness matrix.

What it does now

- **Ply angle:** the angle θ is required. The agent asks for it if it's missing and never assumes 0°. θ is measured from global x to the fibre direction, counter-clockwise about +z.
- **Rotated matrices:** it computes and reports $\bar{Q}$, the 3×3 plane-stress stiffness in order [xx, yy, xy]. It also reports $\bar{C}$, the full 6×6 3D stiffness in order [xx, yy, zz, xy, yz, xz]. Both use engineering shear strains, in MPa.
- **Wrappers:** the material chain is `ElasticOrthotropic → PlaneStress → PlateFromPlaneStress`. `PlaneStress` sets σ33 = 0, and `PlateFromPlaneStress` adds transverse shear. The result goes into a `LayeredShell` section. A new helper, `define_ply_plate_material(p, t3d, tps, tplate)`, builds all three.

Why the rotation is on the element I searched the OpenSeesPy binary for a material or wrapper that takes an angle and found none. `ElasticOrthotropic` can't hold the coupling terms ($\bar{Q}16$, $\bar{Q}26$) that an off-axis ply has. So the material stays in its own fibre axes, and the angle is set on the element with `ASDShellQ4 ... '-local', cos θ, sin θ, 0`. The script prints those exact arguments.

Checks

- With `--check`, the script builds one shell element in OpenSees, applies three membrane strains, and compares the stiffness it measures with $t\cdot\bar{Q}$.
- At 0°, ±45°, 90°, 30° and 17.3° the difference was about 1e-16, which is machine precision.
- Reducing $\bar{C}$ to plane stress also gives $\bar{Q}$ exactly.
- These checks cover membrane (in-plane) stiffness only. I didn't run a separate bending check, though bending uses the same material and orientation.

Limits to know about

- **One angle per element:** each element has a single `-local` orientation, so every layer in its `LayeredShell` gets the same angle. A laminate with plies at different angles needs another step, for example one coincident element per ply sharing the same nodes. I haven't built that. The agent will point out this limit if you ask for a multi-angle layup.
- **One transverse shear modulus:** `PlateFromPlaneStress` accepts only one, but the ply really has two (G13 and G23). It defaults to G13 and can be changed with `--G-out`. For a thin plate this has almost no effect.
- **Negative angles:** pass them as `--theta=-45`, with the `=`, so they're read correctly.

The changes are in `ply_properties.py` and `PlyPropertyGenerator.md`. Both are still uncommitted.

> **You** Fri 23 Oct 2026, 12:11
>
> Now I want to create a multi-layer laminate with different ply orientations. Create a 2nd agent called LayupFromPlyGenerator that takes a user input of the layup (i.e. the angles of the plies and their respective thicknesses), calls the PlyPropertyGenerator agent for each ply to generate the material properties along the correct orientations, and creates a LayeredShell in OpenSeesPy with the plies in order

Claude

I'll start by checking two LayeredShell details that decide the design: which end of the stack is the bottom, and how accurately it integrates bending stiffness through each layer.

- ▸ Bash: Test LayeredShell bending integration and layer order
- ▸ Write: Code for Project\10-23-26 (Tara)\layup.py
- ▸ Bash: Run layup tool and ABD checks on several layups
- ▸ Edit: Code for Project\10-23-26 (Tara)\layup.py
- ▸ Bash: Large-deflection equivalence test of coincident-element laminate
- ▸ Bash: Rerun large-deflection equivalence with relative tolerance
- ▸ Bash: Check if coincident vs reference gap scales with tolerance
- ▸ Bash: Compare linear vs corotational discrepancy
- ▸ Bash: Diagnose source of coincident-element mismatch

- Bash: Read ASDShellQ4/T3 usage strings and options
- Bash: Mesh study: EAS vs -noeas, coincident vs reference
- Edit: Code for Project\10-23-26 (Tara)\layup.py
- Bash: Rerun large-deflection equivalence with -noeas

Claude

[0/90]s now matches to 7e-15. The unsymmetric case only stalls on the very tight 1e-9 tolerance at the last step, so I'm using 1e-7.

- Bash: Rerun large-deflection equivalence, tol 1e-7
- Bash: Test [30]4 convergence with -noeas, both models
- Bash: Mesh study of EAS vs -noeas at large deflection
- Write: .claude\agents\LayupFromPlyGenerator.md
- Edit: Code for Project\10-23-26 (Tara)\layup.py
- Bash: Run layup tool on symmetric quasi-isotropic layup

Claude

Everything checks out. One cosmetic fix: the `-local` axis prints 6.1e-17 instead of 0 for 90° plies.

- Edit: Code for Project\10-23-26 (Tara)\ply_properties.py
- Bash: Recheck both tools after local_axis rounding

Claude

Both tools pass their checks again, including an unequal-thickness [0/90/−30] layup. The last piece is the background mesh study, which shows how much the `-noeas` stiffening costs on finer meshes. I'll report once it finishes.

Background task finished: Background command "Mesh study of EAS vs -noeas at large deflection" completed (exit code 0)

- Bash: Read mesh study results

> **Recap:** LayupFromPlyGenerator and layup.py are written. The laminate matches classical lamination theory, and the per-ply elements match a single laminate element exactly at large deflection. Without enhanced strain modes the element converges more slowly with mesh refinement. Nothing is committed.

Claude

I created **LayupFromPlyGenerator**, plus a new `layup.py` that does the work. It takes ply angles and thicknesses from bottom to top, plus the ply constants. It builds each ply with the PlyPropertyGenerator code and creates `LayeredShell` sections with the plies in order. It also checks the result against classical lamination theory.

The agent doesn't launch PlyPropertyGenerator. In Claude Code a subagent can't start another subagent. LayupFromPlyGenerator therefore runs PlyPropertyGenerator's code for each ply and reads that agent's file for its rules, including the nu23 note.

How the laminate is built

A single `LayeredShell` can't hold plies at different angles. OpenSees can't rotate a material layer by layer, and an ASDShellQ4 element has one `-local` orientation for all its layers. So the tool works around it:

- **One section per ply:** each ply gets its own `LayeredShell` covering the full laminate in order. Its own ply sits at its true z position, and the other plies' slots are zero-stiffness spacers.
- **One element per ply:** `add_laminate_elements` places one ASDShellQ4 per ply on the same four nodes, each oriented at that ply's angle. Together they act as the full laminate, including the bending–stretching and twisting couplings.

In a model script it looks like this:

```
layup = build_layup(make_plies([0, 45, -45, 90], [0.125], p))
for quad in quads:
    ele = add_laminate_elements(ele, quad, layup, '-corotational')
```

Checks

- **Against lamination theory:** I tested unsymmetric [0/45/−45/90], symmetric [0/±45/90]s, unequal-thickness [0/90/−30] and a single ply. The in-plane (A) and coupling (B) stiffnesses match to about 1e-16. The bending stiffness (D) is within 0.4%.
- **Large deflection:** I compared against an independently built single-element model on a cantilever strip with corotational elements. [0/90]s, unsymmetric [0/90/0/90] and off-axis $[30]_4$ all agree to about 1e-14, at tip deflections up to about 60% of the length.

Things you need to know

- `-noeas` **is added by default, and it makes the element stiffer on coarse meshes.** ASDShellQ4's enhanced strain modes don't add up correctly across the per-ply elements: with them on, results were off by 2% on a coarse mesh. Turning them off makes the approach exact. On the $[30]_4$ strip at large load, the gap from the default element was 5.1% at a 20×2 mesh and 2.1% at 40×4. At 80×8 neither version converged with my solver settings, so I don't have that point. Plan a mesh convergence study for your beam.
- **Each ply is split into sublayers.** `LayeredShell` evaluates each layer only at its mid-plane, which underestimates bending stiffness. By default every ply is split so there are at least 15 layers through the thickness, which keeps the bending error near 0.5%. `--n-sub` raises it.
- **Run time grows with the number of plies.** Each facet gets one element per ply.
- **Layer order:** layer 1 is the bottom (−z) ply. I confirmed this from the sign of the bending–stretching coupling.

Small change to PlyPropertyGenerator: `local_axis` now rounds 6e-17 to 0 so the printed axis reads cleanly. Its checks still pass.

Nothing is committed. The new and changed files are `layup.py`, `LayupFromPlyGenerator.md` and `ply_properties.py`.

> **You** Fri 23 Oct 2026, 12:27
>
> Where do these 2 agents live?

Claude

Both agents are in the repo's `.claude/agents/` folder, at the project root:

- `.claude/agents/PlyPropertyGenerator.md`
- `.claude/agents/LayupFromPlyGenerator.md`

Full path: `D:\Users\tvenkata\projects\ME295_project\.claude\agents\`

These are project-level agents. Claude Code loads them for any session opened in `ME295_project` or a subfolder of it, including this one. Once you commit and push, Federico will get them too. If you want them in every project on your machine instead, they'd go in `D:\Users\tvenkata\.claude\agents\`. But they point to this repo's Python files and the `shells` env, so project-level is the right place.

The agent files are only instructions. The code they run is in `Code for Project/10-23-26 (Tara)/`:

- `ply_properties.py`, used by both agents
- `layup.py`, used by LayupFromPlyGenerator

None of this is committed yet. Git shows `.claude/` and `Code for Project/10-23-26 (Tara)/` as untracked.

You Fri 23 Oct 2026, 12:28

If I close this Claude session, will the agents still exist? And can I call them from another Claude session?

Claude

Yes to both. The agents are ordinary files on disk, so closing this session doesn't affect them.

Using them from another session:

- **Any new Claude Code session opened in `ME295_project` or one of its subfolders** loads them automatically when it starts. Ask for one by name, e.g. "Use PlyPropertyGenerator with E1 = 135 GPa, E2 = 10 GPa, nu12 = 0.3, G12 = 5 GPa, theta = 45". Claude can also pick one on its own when your request matches its description.
- **This session doesn't have them loaded.** Agents are read at session start, and these were created mid-session, so start a new session to use them.
- **Sessions opened in a different folder won't see them**, and neither will regular claude.ai chat outside the Code tab. They're project-level agents.

Caveats:

- **They're untracked in git.** Until you commit them, a `git clean` or a fresh clone wouldn't include them. They also won't reach Federico or another machine until you commit, push and pull.
- **The Python path is specific to your machine.** Both agents point to `D:\Users\tvenkata\miniforge3\envs\shells\python.exe`. Anyone else using them would need to change that line, or I can switch it to "the `shells` conda env" so it works on any machine.

You Fri 23 Oct 2026, 12:40

I now want to check that the ABD matrix implied by the definition of the layup in OpenSeesPy matches what I would expect. Please build a 3rd agent called ABDFromOpenSeesPyLayup that takes the calls to OpenSeesPy from the two previously-created agents (PlyPropertyGenerator and LayupFromPlyGenerator) and outputs (in the Claude chat) the ABD matrix of the layup

▸ Read: Code for Project\10-23-26 (Tara)\layup.py

You (sent while Claude was working) Fri 23 Oct 2026, 12:41

Do not use the functions to calculate the ABD matrix that are built into the layup.py function as part of this agent

▸ Edit: Code for Project\10-23-26 (Tara)\layup.py
▸ Bash: Add --emit option to layup.py CLI
▸ Read: Code for Project\10-23-26 (Tara)\layup.py
▸ Edit: Code for Project\10-23-26 (Tara)\layup.py
▸ Bash: Emit OpenSees calls for a test layup

Claude

The exported calls file is plain OpenSees calls for one facet. Now the self-contained ABD module: it reads only those calls and imports nothing from `layup.py` or `ply_properties.py`.

▸ Write: Code for Project\10-23-26 (Tara)\abd_from_opensees.py

Claude

Testing on the emitted [0/45/−45/90] file, then on deliberately broken copies to make sure it catches mistakes: wrong ply angle sign, ν13 in the `vzx` slot, and plies out of order.

- Bash: Run ABD tool on good and broken calls files
- Bash: Test single-ply hand file and non-literal error
- Bash: Rerun single-ply test to see its output
- Edit: Code for Project\10-23-26 (Tara)\abd_from_opensees.py
- Edit: Code for Project\10-23-26 (Tara)\abd_from_opensees.py
- Bash: Retest single-ply, variable error, and good file
- Edit: Code for Project\10-23-26 (Tara)\abd_from_opensees.py
- Edit: Code for Project\10-23-26 (Tara)\abd_from_opensees.py
- Bash: Retest single ply and unequal-thickness layup

> **You (sent while Claude was working)** Fri 23 Oct 2026, 12:44
>
> This agent should not calculate the ABD matrix from classical laminate theory. I will have another agent do that independently

Claude

Understood. I'll take CLT out of this agent completely, so it only measures and prints the ABD that OpenSeesPy implies. Comparing it with CLT is left to your other agent.

- Bash: Locate CLT-related code in ABD module
- Bash: Remove CLT computation from ABD module, add CSV save
- Bash: Review main and rerun the ABD tool without CLT
- Edit: Code for Project\10-23-26 (Tara)\abd_from_opensees.py
- Bash: Rerun ABD tool, check CSV and imports

Claude

The module now outputs only the ABD measured from OpenSees, and imports nothing but numpy and openseespy. Now the agent definition.

- Write: .claude\agents\ABDFromOpenSeesPyLayup.md

Claude

Last, I'm pointing LayupFromPlyGenerator to the new `--emit` option so its output can feed the new agent.

- Edit: .claude\agents\LayupFromPlyGenerator.md
- Bash: Show untracked files and agents

> **Recap:** ABDFromOpenSeesPyLayup is written. It reads the OpenSeesPy calls, measures A, B and D from OpenSees alone (no CLT, nothing from layup.py), and prints them. I tested it on generated, hand-written and deliberately broken layups. Nothing is committed.

Claude

I created **ABDFromOpenSeesPyLayup**. It reads the OpenSeesPy calls for a layup, measures the ABD matrix from OpenSees itself, and prints A, B and D in the chat.

Following your two instructions, it computes nothing from lamination theory and uses nothing from `layup.py` or `ply_properties.py`. The new `abd_from_opensees.py` imports only numpy and openseespy. An earlier draft had a lamination-theory comparison; I removed it when you said another agent will do that.

How it works

- **Input:** a Python file containing the `ops.nDMaterial`, `ops.section` and `ops.element` calls. It reads the file without running it; any other lines, such as node definitions or analysis setup, are ignored.
- **Getting that file:** `layup.py` has a new `--emit FILE` option that writes out the exact calls LayupFromPlyGenerator makes for one element. For calls pasted into the chat or taken from your own script,

the agent writes them to a file itself. They must use literal numbers, though `*[mat, t]*n` is fine. If they use variables, it fills in the values from your script or asks you.
- **Measurement:** it rebuilds the elements on a 1×1 mm square and applies each of the three in-plane strains and three curvatures in turn as exact nodal displacements and rotations. It then gets each ABD entry from the work done by the reactions, so no OpenSees sign conventions are involved.
- **Output:** a table of what OpenSees received for each element (ply angle, thickness, which z range holds the ply, options), then A, B and D with units. It also reports a symmetry check. `--save abd.csv` writes the full-precision matrix to a file, which your comparison agent can read directly.

Testing

- **Generated layups:** I ran it on [0/45/−45/90] and [0/90/−30] with unequal thicknesses. The matrices are symmetric to about 1e-16, and they match the numbers I got earlier by a separate route.
- **Hand-written file:** a single 30° ply in PlyPropertyGenerator's style, with arbitrary node and element numbers, worked.
- **Deliberately broken files:** a flipped ply angle and plies out of order both clearly changed the ABD (27–80% differences), so a comparison would catch them.

What it can't catch

- **Out-of-plane properties:** ν13, ν23, ν31 and G23 have no effect on the ABD. Putting ν13 in the wrong slot gave an identical ABD, so this check can't detect errors in those properties.
- **Slightly low D:** OpenSees's D comes out a little below the exact value, by about 1/(number of layers)² (around 0.4% with the default sublayers). That's how `LayeredShell` integrates through the thickness. The agent reports it as the D OpenSees will actually use, so your comparison agent will see it as a small gap in D only.

The agent file is `.claude/agents/ABDFromOpenSeesPyLayup.md`. I also updated LayupFromPlyGenerator's instructions to mention `--emit`. As with the other two, you'll need a new session to use it, and nothing is committed yet.

> **You** Fri 23 Oct 2026, 13:20
>
> Now create another agent, called TheoreticalABDGenerator, which takes in the same material properties as PlyPropertyGenerator and the same layup information as LayupFromPlyGenerator (i.e. angles and thicknesses), and uses classical laminate theory to output the ABD matrix into the Claude chat window

Claude

I'll write this as a self-contained module that imports neither `layup.py` nor `ply_properties.py`, keeping it independent like the OpenSees-side check. For a third independent route, it uses the Tsai–Pagano invariant form of $\bar{Q}$ instead of the matrix rotations used elsewhere.

> **You** Fri 23 Oct 2026, 13:20
>
> Do not have this agent interface with the other agents

> **You** Fri 23 Oct 2026, 13:20
>
> Or Python scripts

> **Claude asked:** Can TheoreticalABDGenerator run its own new standalone Python file, or should it use no Python at all?
>
> - **Own standalone script (Recommended)**: A new file using only numpy, written fresh. It won't import or call layup.py, ply_properties.py, abd_from_opensees.py or any agent. Exact arithmetic.
> - **No Python at all**: The agent does the CLT arithmetic itself, by reasoning, and writes the ABD in chat. Fully separate, but hand arithmetic on 6x6 sums can have rounding or arithmetic slips.

Your answer: Own standalone script (Recommended)

Claude

I'll write a fresh standalone script for it. The agent won't call the other agents or their scripts.

- ▸ `Write: Code for Project\10-23-26 (Tara)\theoretical_abd.py`
- ▸ `Write: .claude\agents\TheoreticalABDGenerator.md`
- ▸ `Bash: Test CLT script: hand check and cross-check with OpenSees ABD`

> **Recap:** TheoreticalABDGenerator is written. It computes the ABD from lamination theory with its own numpy-only script and doesn't touch the other agents or scripts. It matches a hand calculation and the OpenSees-measured ABD. Nothing is committed.

Claude

I created **TheoreticalABDGenerator**. It takes the same inputs as the other two agents: E1, E2, nu12 and G12 (nu23 optional), plus ply angles and thicknesses from bottom to top. It computes the ABD with classical lamination theory and prints A, B and D in the chat.

It's fully separate from the rest

- It runs only its own new script, `theoretical_abd.py`, which imports just numpy.
- Its instructions forbid calling the other agents, running or reading their scripts, and using OpenSeesPy. It's only allowed the Read and Bash tools.
- The rotated ply stiffnesses come from the Tsai–Pagano invariant formulas. That's a third, different route from the matrix rotation in `ply_properties.py` and the explicit formulas I'd briefly put in the ABD agent.

What it reports

- The ply stiffnesses Q11, Q22, Q12 and Q66, and a layup table with each ply's z range (ply 1 at the bottom).
- A, B and D with units and the sign convention. It also points out features such as B = 0 for a symmetric layup.
- `--save abd_clt.csv` writes the matrix in the same CSV layout as ABDFromOpenSeesPyLayup's `--save`, so your comparison step can load both files the same way.
- nu23 is accepted to keep the inputs the same, but the output says it's unused, because lamination theory is plane stress.

Checks I ran in this session (the agent itself never compares)

- **By hand:** a single 0° ply of 0.2 mm gives A11 = Q11·t = 27181.2 N/mm and D11 = Q11·t^3/12 = 90.604 N·mm. Both match.
- **Against OpenSees:** for [0/90/−30] with thicknesses 0.1/0.15/0.2 mm, I compared its CSV with the one ABDFromOpenSeesPyLayup measured. A and B match to about 1e-15. D differs by 0.37%. That's the expected `LayeredShell` layer-integration effect: OpenSees's D runs a little low, by about 1/(number of layers)^2. Your comparison should therefore expect A and B to agree exactly and D to agree within about 0.5% at the default sublayers.
- **Bad input:** a negative thickness gives a clear error.

The agent file is `.claude/agents/TheoreticalABDGenerator.md`. Like the others, it needs a new session to use, and nothing is committed yet.

> **You** Fri 23 Oct 2026, 14:40
>
> Write a plain-text summary file of the work I did today, both in this Claude session and the one called "Python environment setup for ME 295 project" in which I set up the agents. I would like to record what I did and the major takeaways for future work so that Federico (and my future self) can use them

- list sessions
- list events
- get session
- list events

Claude

I've read the environment-setup session. Next I'll look at how the existing running logs and Federico's summary are laid out.

- Bash: Read running logs and Federico's summary format
- Write: Code for Project\10-23-26 (Tara)\SUMMARY.txt

Claude

I'll confirm the extra files really came from that third session before leaving the attribution in the summary.

- Bash: Check timestamps and headers of extra files

Recap: I wrote SUMMARY.txt in the 10-23-26 (Tara) folder. It covers the environment-setup session and this agent-building session, with the major takeaways and future work. Nothing is committed.

Claude

I wrote the summary to `SUMMARY.txt`, in plain text, next to today's code.

One correction to your description: the agents were built in this session ("OpenSeesPy CFRP ply property generator"). The "Python environment setup for ME295 project" session was the Python install and re-running Federico's models. The summary covers both.

What's in it

- **Part 1, environment:** what was installed and where, with versions. It explains the `conda-launchers` fix needed to create the environment, and how to run scripts. It also records the results of re-running Federico's four scripts, which all reproduce his README, including the large-deflection run stalling at α = 6.817. It notes the one side effect: a new `smoke_tests/gui_check.png` in his folder.
- **Part 2, agents:** a table of the four agents and the Python file each one runs, with how to use them. It notes that agents need a new session and that the Python path is hard-coded to your machine, which Federico will need to change. There's a short description of each agent and a list of what was verified.
- **Major takeaways:** 13 numbered points, for example:
  - OpenSees's `vzx` slot takes ν31, not ν13.
  - OpenSees can't rotate a material, so the angle goes on the element with `-local`.
  - Multi-angle laminates use one element per ply, which requires `-noeas`; that makes coarse meshes about 2–5% stiffer, so a mesh convergence study is needed.
  - `LayeredShell` integrates each layer at its mid-plane only, so its D runs slightly low.
  - A and B should match theory to round-off, and D within about 0.5%.

- **Future work:** commit and push, make the Python path portable, and write an ABD comparison step. Then run the mesh study and move on to the real curved beam geometry, plus a couple of optional improvements.

The folder also has seven files (`layup_90_0.py`, `layup_L1…L4`, `ply_materials_*.py`) from a third session today, `CFRP layup with 0/90 plies`. Their timestamps match that session. You didn't ask me to cover it, so the summary only lists them as not covered.

Nothing is committed yet. I didn't add an entry to `Running Log/Log Before Lab 2 Deadline.txt`.

> **You** Fri 23 Oct 2026, 14:44
>
> Yes, I also want to summarize the session "CFRP layup with 0/90 plies" in that same summary document

▸ list events
▸ Bash: Extract user turns from third session transcript
▸ list events
▸ Bash: Read CLT result and new workflow request
▸ Bash: Read main-session replies for the four-layup workflow

Claude

I've read the 0/90 session. Next I'm skimming the headers of the seven files it created so the summary describes them accurately.

▸ Bash: Read headers of files from the 0/90 session

Claude

Now adding Part 3 to the summary, plus the extra takeaways it turned up.

▸ Edit: Code for Project\10-23-26 (Tara)\SUMMARY.txt
▸ Edit: Code for Project\10-23-26 (Tara)\SUMMARY.txt
▸ Edit: Code for Project\10-23-26 (Tara)\SUMMARY.txt
▸ Edit: Code for Project\10-23-26 (Tara)\SUMMARY.txt
▸ Edit: Code for Project\10-23-26 (Tara)\SUMMARY.txt

Claude

I'll check two of the numbers quoted from that session against `theoretical_abd.py` before finishing.

▸ Bash: Spot-check L3 A16 and L1 D11 from the summary

Recap: SUMMARY.txt now covers all three sessions, including the [90/0] laminate and the four 4-ply layups, with six more takeaways. I spot-checked the quoted numbers. Nothing is committed.

Claude

I added the "CFRP layup with 0/90 plies" session to `SUMMARY.txt` as a new **Part 3**, and it's now listed in the header with the other two sessions.

What Part 3 records

- **The workflow:** the main session ran the ply, layup and OpenSees-ABD agents in sequence and the lamination-theory agent independently, then compared the two ABDs.
- **[90/0] laminate** (E1 = 140 GPa, 1 mm plies): the material inputs and assumptions, the two files it created, and the results. A and B match exactly, including the expected B11 = −B22 coupling for an unsymmetric cross-ply. Every D term is low by exactly 1/256, which is 0.39% from the 16 layers through the thickness.
- **Four 4-ply layups** (E1 = 139.5 GPa, 54 µm plies): [0/90/90/0], [90/0/0/90], [45/0/0/45] and [−45/90/90/−45], with their five files.
  - B is zero for all four.

    - D is within 0.06% for the cross-plies and 0.17% for the ±45° layups, with OpenSees lower.
    - It explains why the first two layups have the same A but different D.
    - It records the coupling terms in the ±45° layups, and that the last layup is the third rotated by 90°.

New takeaways (14–18)

- **Plies at the same angle can share one element.** That session found this halves the element count, and it's exact with `-noeas`. `layup.py` doesn't do it yet, so it's also listed under future work.
- **Compute ν21 from ν12·E2/E1** rather than using a separately rounded value, so the constants stay consistent.
- **Keep material tags from clashing** between files loaded into the same model.
- **Check the agents' wording as well as their numbers.** One ABD agent report labelled the Poisson's ratio slots wrongly, though the model itself was correct.
- **Why the per-layup scripts needed their calls recorded** before the ABD agent could read them.

Checks and caveats

- I reran `theoretical_abd.py` on two of the quoted numbers, A16 for [45/0/0/45] and D11 for [0/90/90/0]. Both match.
- That session saved its ABD CSV files only in its temporary scratch folder, not in the project. The summary says so, and notes they can be regenerated with `--save`.

The paragraph that said this session wasn't covered has been removed. Nothing is committed.

> **You** Mon 26 Oct 2026, 09:35
>
> Create a PDF of this chat

▸ Skill: anthropic-skills:pdf
▸ Bash: Locate session transcript and check reportlab
▸ Bash: Check for Edge and Markdown/PDF Python modules
▸ Bash: Inspect transcript JSONL entry types
▸ Bash: List user messages and mid-turn messages
▸ Write: scratchpad/chat_to_html.py
▸ Bash: Render transcript to HTML